Disciplina: Sistemas Distribuídos
Professora: Ana Cristina Barreiras Kochem Vendramin

Avaliação (valor 2,5)
Replicação de dados. PyRO.

Implemente uma **replicação por quórum** em um *cluster* de *brokers*. Utilize uma versão simplificada do KRaft (Kafta Raft) para a gestão dos dados e coordenação do cluster e a *middleware* **PyRO** (*Python Remote Objects*) para prover a comunicação entre os processos.

KRaft é modo de operação do Apache Kafka que substitui o uso do ZooKeeper para a gestão de metadados e a coordenação do cluster. Em vez de depender de um sistema externo, KRaft implementa o protocolo de consenso Raft para gerenciar as operações de cluster, como a configuração de tópicos, a replicação de dados e a eleição de líderes. O Kafta utiliza um protocolo binário próprio. Neste trabalho, será utilizado o PyRO para a comunicação entre os processos.

A **replicação por quórum de *brokers*** (subconjunto do total de *brokers*) serve para garantir a alta disponibilidade e a resiliência do sistema. Ela permite que um *cluster de brokers* continue a operar mesmo se algum *broker* falhar, assegurando que haja cópias suficientes de dados e metadados. Nesse modelo de replicação, um único líder recebe gravações dos publicadores e, em seguida, tenta replicá-las para outros *brokers* que atuam como seguidores. Porém, em vez de esperar que todas as réplicas confirmem a gravação, o líder espera apenas a confirmação da maioria das réplicas, incluindo a si mesmo. Isso é chamado de quórum. Depois que a maioria dos seguidores confirmam a replicação da gravação, o líder a considera comprometida (*commit*) e retorna uma confirmação/*acknowledge* para o cliente publicador.

Um *broker* é um servidor que armazena dados e processa requisições de leitura e escrita. Um *broker* em um *cluster* tem uma das seguintes funções: **líder, votante** (seguidor) **ou observador**. O líder e votantes formam o quórum e são responsáveis por manter o *log* replicado em consenso e por eleger novos líderes quando necessário.

Um dos *brokers* do *cluster* atua como **líder**, responsável pela coordenação das operações do *cluster*, como a criação de tópicos, gerenciamento de partições e controle de replicação. O líder registra novos *brokers*, detecta falhas de *brokers* e lida com todas as solicitações que alteram o *log* de metadados do *cluster* (como a configuração de tópicos, partições e *brokers*)**.** Após uma gravação, o líder notifica os votantes para que busquem os dados atualizados.

Os ***brokers* votantes** fazem parte do quórum, participando ativamente do processo de replicação e consenso. Eles armazenam *logs* de metadados e réplicas de partições de dados, garantindo a consistência e a disponibilidade das gravações. As réplicas precisam estar atualizadas e em consenso para que uma gravação seja considerada comprometida.

Todos os outros *brokers* no *cluster* se comportam como **observadores** que apenas leem o *log* replicado e confirmado dos votantes para se atualizarem em relação ao quórum. Esses observadores não fazem parte do quórum, isto é, não participam ativamente da replicação ou confirmação de gravações.

Para tolerar f falhas consecutivas, é preciso um **quórum de replicação de tamanho 2f+1**.

Cada registro adicionado ao *log* está associado à **época do líder**. Uma **época de líder** refere-se a um número sequencial que é associado a um líder em um *cluster* de *brokers*. Esse número é utilizado para identificar e diferenciar as várias instâncias de liderança ao longo do tempo. Quando um novo líder é eleito, sua época é incrementada, assegurando que haja um único líder ativo em um determinado período

Diferente do RabbitMQ que utiliza o modelo *push*, no Kafta os consumidores buscam mensagens ativamente no *broker* (**modelo *pull***). Consumidores enviam solicitações para ler mensagens de um tópico, o que permite um controle mais flexível, pois os consumidores podem escolher o ponto inicial de leitura (*offset*, isto é, um inteiro que indica a posição única e sequencial de uma mensagem dentro de uma partição de um tópico) e controlar o ritmo de leitura, podendo até reprocessar mensagens antigas se necessário.

Cada **tópico** é dividido em **partições**, que são as unidades de paralelismo e armazenamento. Cada partição é replicada em múltiplos *brokers,* chamados de **réplicas**, para garantir disponibilidade. Cada partição é representada como um ***log* de mensagens**, que são armazenadas em ordem de chegada. Os instantâneos locais de metadados são versionados pelo ***log offset***, e é fácil avaliar a obsolescência dos dados e corrigir quaisquer potenciais divergências. Cada registro adicionado ao *log* está associado à época do líder.

Requisitos:

1. **(0,5) Inicialização, envio e consumo de dados.**
    1. Considere, pelo menos, um *cluster* de 4 processos *brokers*: 1 líder, 2 processos votantes e 1 processo observador;
    2. Considere 1 processo publicador (produtor) e 1 processo consumidor, 1 tópico e 1 partição;
    3. Cada *broker* possuirá seu ID, seu log, seu estado (líder, votante ou observador);
    4. Não precisa implementar o processo de eleição de líder. O primeiro *broker* a ser executado será o líder. Esse *broker* irá se registrar no serviço de nomes com o nome Líder-Epoca1. Não

vamos assumir falha desse líder e, portanto, não haverá reeleição;

5. O líder e os votantes conhecem o número total de membros no quórum;
6. Suponha que cada processo saiba qual será sua função (estado). Ao inicializarem, os *brokers* votantes e observadores vão buscar a referência do líder no serviço de nomes e vão se registrar no líder passando sua referência e seu estado;
7. O publicador vai interagir com a aplicação para registrar dados;
8. O consumidor vai interagir com a aplicação para consumir dados.

2. **(1,0) Replicação dos dados**:
    1. O líder recebe uma gravação e a adiciona ao seu log;
    2. O líder notifica os votantes para que busquem os dados atualizados;
    3. Os votantes, ao receberem a notificação, enviam uma requisição de busca ao líder, incluindo a época de onde querem buscar os dados (isto é, época do último *offset* que possuem em seu *log* local) e o *offset* de busca;
    4. Ao receber uma requisição de busca, o líder verifica se o *offset* de busca e a época de busca são consistentes com seu próprio *log*. Se não coincidirem, a resposta da busca indicará um erro e informará a maior época e seu *offset* final antes da época solicitada. Se a solicitação for válida, ele retornará os dados correspondentes a partir do *offset* fornecido na requisição;
    5. Pode ocorrer de um votante divergir do líder em relação às entradas do *log*. É responsabilidade dos votantes detectar mudanças de líder e usar as respostas de busca para truncar (isto é, remover) entradas antigas de seu *log* que não foram confirmadas, se necessário, de modo a se manter consistente com o estado do sistema.
        - ♦ Por exemplo, um votante que era um antigo líder em uma época anterior pode possuir alguns registros adicionados ao seu *log* local que não foram replicados para a maioria do quórum e, portanto, não foram confirmados. Ao receber a solicitação, o líder descobrirá que a época e *offset* fornecidos não coincidem, e, portanto, retornará um código de erro na resposta, informando ao votante que a época x solicitada possui apenas registros confirmados até o *offset* y. O votante, então, truncará seu *log* local até o *offset* y e reenviará a solicitação de busca, desta vez com a época x e *offset* y.
    - O votante recebe os dados e os adiciona ao seu log, enviando uma confirmação de volta ao líder;
    - O líder registra a confirmação e verifica se o quórum de votantes confirmou a gravação. Se sim, a gravação é marcada como comprometida e uma confirmação é retornada ao publicador.

3. **(1,0) Tolerância a falhas dos votantes:**
    - Os *brokers* votantes enviam periodicamente mensagens de "*heartbeat*" para o líder, indicando que estão disponíveis. Isso permite que o líder monitore a saúde dos *brokers* registrados;
    - O líder verifica a disponibilidade de cada votante através da checagem de *heartbeats* enviados pelos votantes em intervalos regulares. Se o líder não receber um *heartbeat* de um votante dentro de um intervalo de tempo pré-definido, ele considera esse votante como indisponível ou falho. Nesse caso, é verificado se o número de votantes ativos é menor que o tamanho do quórum necessário;
    - Se o número de votantes disponíveis for inferior ao tamanho mínimo do quórum, o líder precisará redistribuir a partição que estava sob a responsabilidade desse votante que falhou para garantir que haja cópias suficientes dos dados em outros votantes. Um observador será promovido a votante para garantir que haja um quórum suficiente para manter a operação do *cluster*;
    - O novo votante deverá solicitar dados ao líder para garantir que possui a versão mais recente dos dados.
    - O líder notificará todos os votantes sobre a inclusão do novo votante.

<u>Observações</u>:

- Desenvolva uma interface com recursos de interação apropriados.
- É obrigatória a defesa da aplicação para obter a nota.
- O desenvolvimento do sistema pode ser individual ou em dupla.

Referência:

Guozhang Wang. “Why ZooKeeper Was Replaced with KRaft – The Log of All Logs”. 2022. Disponível em: https://www.confluent.io/blog/why-replace-zookeeper-with-kafka-raft-the-log-of-all-logs/